

WebRTC (Web Real-Time Communication) 정리

1. 기본 개념

- WebRTC는 웹 브라우저 간에 플러그인 없이 실시간 통신을 가능하게 하는 API입니다.
- P2P 기반으로 영상/음성 통화, 파일 공유 등을 지원하며, W3C와 IETF에서 표준화되었습니다.

2. WebRTC 아키텍처

- 애플리케이션 계층: JavaScript와 Web API를 사용하여 WebRTC 기능을 구현합니다.
- WebRTC API 계층: PeerConnection, MediaStream 등을 통해 미디어와 데이터 전송을 관리합니다.
- 전송 계층: ICE, STUN, TURN을 통해 네트워크 연결을 설정하고 관리합니다.

3. 핵심 컴포넌트

3.1 미디어 캡처 및 스트림

- **MediaStream API**: getUserMedia()로 카메라와 마이크에 접근하고, getDisplayMedia()로 화면을 공유합니다.
- 트랙 제어 기능을 통해 음소거 및 비디오 ON/OFF를 관리할 수 있습니다.

3.2 연결 설정

- **RTCPeerConnection**: 피어 간의 연결을 관리하며, ICE 프레임워크를 통해 네트워크 경로를 설정합니다.
- 미디어 협상은 SDP(Session Description Protocol)를 통해 이루어집니다.

3.3 데이터 채널

- **RTCDataChannel**: 임의의 데이터를 전송하며, SCTP 프로토콜을 사용합니다.
- 게임, 파일 전송 등에 활용됩니다.

4. 시그널링 프로세스

- 시그널링은 피어 간의 연결 설정을 위한 정보 교환 과정으로, WebSocket, XMPP, SIP 등을 사용하여 구현됩니다.

```
const pc = new RTCPeerConnection(configuration);

// Offer 생성 (발신측)
const offer = await pc.createOffer();
await pc.setLocalDescription(offer);
// 시그널링 서버로 offer 전송

// Answer 생성 (수신측)
const answer = await pc.createAnswer();
await pc.setLocalDescription(answer);
// 시그널링 서버로 answer 전송
```

5. 미디어 서버 아키텍처

5.1 Mesh 토폴로지

- 모든 피어가 직접 연결되며, 소규모 회의에 적합합니다.

- 네트워크 부하가 증가할 수 있습니다.

5.2 MCU (Multipoint Control Unit)

- 중앙 집중식으로 미디어를 혼합하여 단일 스트림으로 전송합니다.
- 높은 서버 리소스가 필요합니다.

5.3 SFU (Selective Forwarding Unit)

- 선택적으로 스트림을 전달하여 확장성이 좋습니다.
- Mediasoup, Janus 등이 대표적입니다.

6. 네트워크 통신

6.1 NAT 통과

- NAT 뒤의 클라이언트는 STUN과 TURN을 통해 공인 IP를 얻고, 피어 간의 연결을 설정합니다.

6.2 프로토콜 스택

- 미디어 전송: RTP/RTCP
- 보안: DTLS, SRTP
- 데이터: SCTP
- NAT 통과: ICE, STUN, TURN

7. 성능 최적화

7.1 비디오 최적화

```
const constraints = {
  video: {
    width: { ideal: 1280 },
    height: { ideal: 720 },
    frameRate: { max: 30 },
    codec: { ideal: "VP8" }
  }
};
```

7.2 대역폭 관리

- 적응형 비트레이트, 네트워크 상태 모니터링, 동적 해상도 조정을 통해 최적화합니다.

7.3 코덱 선택

- VP8: 기본 코덱
- VP9: 더 나은 압축률
- H.264: 하드웨어 가속
- AV1: 차세대 코덱

8. 보안

8.1 암호화

- DTLS로 모든 데이터를 암호화하고, SRTP로 미디어 스트림을 보호합니다.

8.2 인증

- ICE 프레임워크와 DTLS 인증서를 통해 피어를 인증합니다.

9. Mediasoup

- Mediasoup는 WebRTC에서 다중 P2P 연결을 지원하는 SFU입니다.
- Node.js 기반으로 동작하며, Worker와 Router 구조를 통해 확장성을 제공합니다.
- 클라이언트는 Transport 객체를 통해 미디어와 데이터를 송수신합니다.

10. NAT Traversal

- STUN과 TURN을 통해 NAT 뒤의 클라이언트가 서로 연결할 수 있도록 합니다.
- STUN은 공인 IP를 얻는 데 사용되며, TURN은 직접 연결이 불가능할 때 중계 서버로 사용됩니다.

11. SDP (Session Description Protocol)

- SDP는 스트리밍 미디어의 초기화 인수를 기술하는 포맷으로, WebRTC에서 세션 프로파일을 묘사하는 데 사용됩니다.

12. ICE (Interactive Connectivity Establishment)

- ICE는 두 단말이 서로 통신할 수 있는 최적의 경로를 찾도록 도와주는 프레임워크입니다.

13. DTLS (Datagram Transport Layer Security)

- DTLS는 UDP 기반 통신에서 TLS를 확장한 프로토콜로, WebRTC에서 데이터를 보호하는 핵심 기술입니다.

Document

- WebRTC API <https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia>
- Mediasoup API <https://mediasoup.org/documentation/v3/mediasoup/api/>
- Mediasoup Design <https://mediasoup.org/documentation/v3/mediasoup/design/>
- <https://skatpdnjs.tistory.com/107>
- <https://olive-su.tistory.com/377>